Cascaded YOLO-based approach for Bangladeshi license plate identification and number recognition

What the paper does, what our Kaggle run found, how it fits our research, and what to show and say to the faculty

Prepared for MD. Rokib Hasan Oli (paper reader) · Supervisor: Dr. Md Mansur ul Mulk · 19 October 2026
Paper: P. Sarker and S. M. Shamsul Alam (Khulna University), *PLOS One* 21(8): e0355858, published 10 Aug 2026, DOI 10.1371/journal.pone.0355858, 42 pages, open access (CC BY 4.0). Page numbers below are the journal pages.
Code and data: github.com/prosensarker4/BLPD (weights, data, a validation notebook; no training script).
Our run: Kaggle notebook `paper02_kaggle.ipynb` on Tesla T4 x2, 3.3 h, all jobs finished (log in `After train/paper02-r.log`).

The whole report on one page

The paper in three lines. Three YOLOv12 models run one after another: a COCO model finds the vehicle, a plate model finds the plate inside the vehicle, and a 104-class model finds every digit, letter, "Metro" and whole district name inside the plate. Sorting those boxes left to right gives the plate text, so no OCR engine is needed. On 150 external photos it reads 95.33 % of plates exactly (143 of 150), against 15.33 % for an OCR pipeline (p.34, p.38).

What our run found. (1) **The paper's number reproduces on its own photos:** the authors' own weights read 95.14 % of their external photos exactly in our run (paper: 95.33 %). (2) **It does not hold at night:** on 60 night photos the same weights read only 38.3 %; our model, trained on the merged, de-duplicated 498R data, reads 63.3 %. (3) **The vehicle stage is not needed:** skipping it changes accuracy by one photo out of 204 but makes the pipeline 2.7× faster (85.8 → 31.5 ms per photo on a T4). (4) On the 204 end-to-end test photos (day and night) our cascade reads 83.3 % of plates exactly and gets 92.6 % of characters right, so it already meets our proposal's 90 % character target in daylight.

Is the training OK? Yes. All 5 jobs ended with `code=0` inside their time caps. Before you quote anything, read the four warnings in section 12. In short: the "7.30 FPS" row is not a real speed, the plate detector's mAP50-95 of 46.2 needs a check, our test data is different from the paper's, and the outputs zip was not fully downloaded.

Why it matters for us. It is the strongest public Bangla baseline, peer reviewed and with weights, and Stage 2 of our proposal (the character-level YOLO) follows its design. Our project fixes exactly the weak points it admits itself (p.41): night, weather, blur, class imbalance and speed.

What to tell the faculty in one sentence. "We reproduced the best published Bangla system to within 0.2 points on its own photos, showed that it fails at night and wastes time on an unnecessary vehicle stage, and our pipeline keeps its good idea and fixes those problems."

How this report is organised

Part	Sections	Read it when
A. Understand the paper	1-7: problem, terms, data, method, results, critique, what breaks on our plates	before you explain the paper to anyone
B. Our experiment	8-12: what we ran, results vs paper, four findings, health check	before you quote any of our numbers
C. Use it	13-19: research use, benefits, faculty slides, script, Q&A, next steps, GitHub	before the faculty meeting

1. What problem does the paper solve?

Bangladesh had 5,759,353 registered motor vehicles by 30 June 2023, 72 % of them motorcycles (BRTA, p.2). Traffic police, toll gates and parking need to read plates automatically. A Bangladeshi plate has four required parts: a **city (district) code** written as a Bangla word, a **vehicle-type letter**, a **two-digit class number** and a **four-digit registration number**. Plates of metropolitan areas add the word **"Metro"** after the city (p.2, Fig. 2).

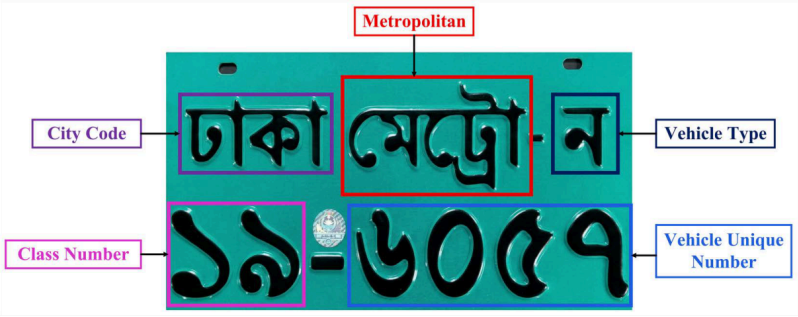


Figure 1. Parts of a standard Bangladeshi plate: city code, "Metro", vehicle type, class number and unique number (paper Fig. 2, p.3). Figure from Sarker & Alam (2026), PLOS One, CC BY 4.0.

Earlier Bangla systems had two problems (p.4-5, Table 1):

- **Too few classes.** Character recognizers used 14 to 60 classes. Real plates need more than 100 (10 digits, about 30 letters, "Metro" and 64 district names).
- **General OCR engines fail on Bangla.** Tesseract and EasyOCR misread conjunct consonants (*jukto borno*) and the two-line layout. Reported OCR accuracy was 78 % (EasyOCR) and 22-73 % (Tesseract).

The paper's answer: treat every piece of the plate, even a whole district name, as an **object** that a YOLO detector can find. Then no OCR engine is needed (p.5).

2. Key terms in plain words

Term	Meaning
Cascade	models run one after another; each one only looks inside the crop given by the one before
LPD / LPNR	License Plate Detection model (finds the plate) / License Plate Number Recognition model (finds the characters)
OCR-free	the text comes from a detector's class labels, not from an OCR engine such as Tesseract or EasyOCR
Class / token	one thing the detector can name: a digit, a letter, "Metro" or a district word such as "Dhaka"
Precision / Recall	of all boxes the model drew, how many were right / of all real objects, how many it found
mAP50	mean average precision when a box counts as correct if it overlaps the true box by at least 50 % (IoU 0.5)
mAP50-95	the same, averaged over stricter overlaps 0.50, 0.55 ... 0.95; it rewards *tight* boxes
Full-number exact match	the whole plate text is right; one wrong character = wrong plate
Character-wise accuracy	share of characters that are right
Levenshtein similarity	1 - (edits needed to turn the prediction into the truth / length of the truth); a nearly right plate scores near 100 %
External validation	a test on photos that were never used for training or tuning
Latency	time to process one photo (ms); FPS = 1000 / latency
Near-duplicate	a different file that shows the same photo or plate (Roboflow copy, re-post, next video frame); see paper #23

3. The data

Set	Images	Train / val / test	Classes	Boxes	Page
Dataset-01 (plates, for LPD)	2,140	1,502 / 425 / 213	1	2,251 (Fig. 3 shows 1,561)	p.6-7, Table 2
Dataset-02 (plate crops, for LPNR)	7,217	5,058 / 1,438 / 721	104	63,879	p.6-10, Tables 2-3
External validation	150	test only	-	-	p.32

- **Sources:** their own phone photos in Khulna (Gollamari, Sonadanga, Zero Point), all in **daylight and sunny weather**, plus web crawling and Roboflow (p.6).
- **Split:** 70 / 20 / 10, done by image. No de-duplication step is described.
- **Strong imbalance:** digit "1" has 7,396 boxes, "Metro" 6,292, "Dhaka" 4,947, but Munshiganj has 2, Nowabganj 11, Bagerhat 37, Joypurhat and Kustia 45 (p.9).
- **Class list:** some "different" classes are the same token written two ways: "Cox-s Bazar" (47) and "Cox-s-Bazar" (58), "Da" (402) and "DA" (141), "Tha" (405) and "THA" (179) (Table 3, p.10). So 104 classes are really about 101 tokens. Our cleaned vocabulary has 101.

4. The method

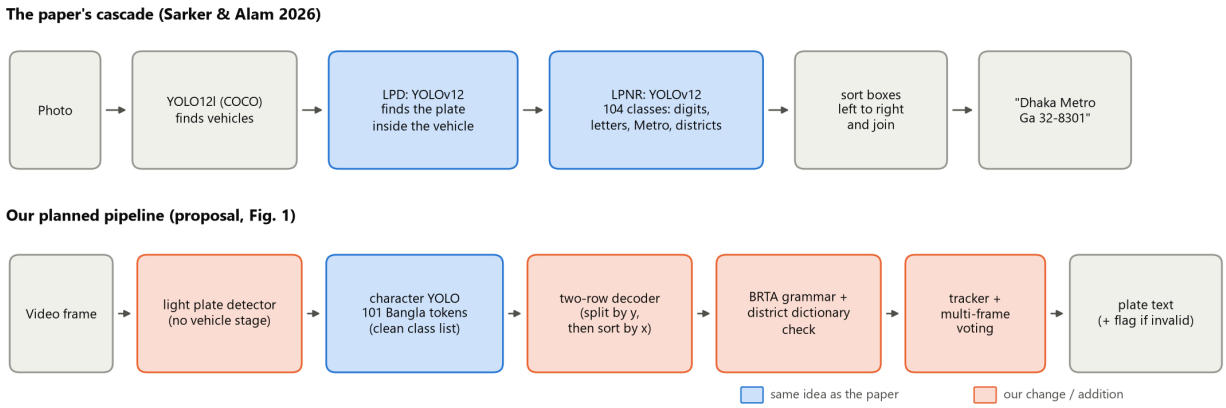


Figure 2 (drawn for this report). Top: the paper's cascade (Algorithm 1, p.15-16). Bottom: our planned pipeline from the proposal. Blue = idea we keep, orange = what we change or add.

Stage	What it does	Model and settings
1. Vehicle	finds cars, buses, trucks and motorcycles; each vehicle box is cropped	Ultralytics YOLO12l pretrained on COCO, not retrained (26.4 M parameters, 88.9 GFLOPs) (p.15)
2. Plate (LPD)	finds the plate inside the vehicle crop; the box is mapped back to the photo	YOLOv12 trained on Dataset-01, 100 epochs (p.15)
3. Characters (LPNR)	finds digits, letters, "Metro" and district words in the plate crop	YOLOv12 trained on Dataset-02, 70 epochs (p.16)
4. Decoding	sort the character boxes from left to right and join the labels: "Dhaka Metro Ga 328301"	simple rule, no language model (p.16)

Training settings (Table 5, p.18): 640×640 input, batch 16, optimizer "auto", lr0 0.01, weight decay 0.0005, NMS IoU 0.7, data augmentation "No", no cosine schedule, deterministic, no early stopping. Training ran on a Colab Tesla T4: LPD 2.42 h (87.3 s per epoch), LPNR 4.35 h (223.6 s per epoch) (p.17-18, p.24). The model size (n, s, m...) is not stated; the repo's files suggest a medium LPD (40.8 MB) and a nano LPNR (5.7 MB).

Why YOLOv12? It adds attention ("area attention" and R-ELAN blocks) to YOLO and was the most accurate YOLO for its speed when the work was done (p.13). On their character data it beat YOLOv8, v9, v10 and v11 (Table 14, p.37).

5. Every result in the paper

Result	Value	Page
LPD validation (425 images)	P 0.993, R 0.970, mAP50 0.974, mAP50-95 0.674	p.21, Table 6
LPD test (213 images, 226 plates)	**P 0.986, R 0.965, mAP50 0.963, mAP50-95 0.685**; 219 found, 7 missed, 6 false	p.22-23, Table 7
LPNR validation (1,438 images)	P 0.952, R 0.905, mAP50 0.950, mAP50-95 0.770	p.24, Table 8
LPNR test (721 images, 6,292 boxes)	**P 0.950, R 0.898, mAP50 0.949, mAP50-95 0.767**; digits mAP50 ≥ 0.985; weakest "Sa" 0.346, Joypurhat 0.42, "Pa" 0.60 (mAP50-95)	p.25-31, Table 9
YOLO versions on Dataset-02 (test mAP50)	v8 0.936, v9 0.938, v10 0.897, v11 0.931, **v12 0.949**	p.37, Table 14
External 150 photos	**plate found 100 %, full number exact 95.33 % (143/150), characters 98.82 %, Levenshtein 99.57 %**	p.34, Table 10
OCR pipeline on the same 150	23 correct = 15.33 % (counted "ignoring minor mismatches")	p.37-38, Table 15
Laptop CPU (i5-8265U), per photo	mean 2,311.53 ms (vehicle 911 + plate 1,091 + characters 233), max 6,438 ms	p.35, Table 11
Desktop GPU (RTX 3050), per photo	mean 117.16 ms (39.6 + 45.2 + 19.5) ≈ 8.5 FPS, max 284.31 ms	p.35, Table 12
Jetson Orin Nano, CPU only, per photo	mean 10.37 s, max 25.44 s	p.36, Table 13



Figure 3. The 7 external photos the cascade read wrongly; the paper calls every error partial, and the plate was found in all of them (paper Fig. 30, p.34). Figure from Sarker & Alam (2026), PLOS One, CC BY 4.0.

6. Critical review

Strengths

- **Peer reviewed and recent** (PLOS One, Aug 2026), with data, weights and a validation notebook public. That is rare for Bangla LPR.
- **Good idea:** word-level classes for district names avoid the hardest OCR problem (conjunct letters), and 104 classes cover nearly every plate.
- **Several end-to-end metrics** (exact, character-wise, Levenshtein) and **per-stage latency on three devices**, with mean, std, min and max.
- **Honest about limits:** the conclusion names low light, weather, blur, occlusion, imbalance and lightweight models as future work (p.41).
- **Its numbers reproduce on its own data:** we confirmed Table 10 (section 10) and Tables 6, 7 and 9 (local retraining, section 9).

Weaknesses (with page numbers, so you can show them)

#	Problem	Why it matters
1	**Test sets leak.** Split by image from Roboflow/web sources with no de-duplication. Our audit: 94 of the 213 LPD test photos (44.1 %) and 270 of the 721 LPNR test crops (37.4 %) have a near-duplicate in their own train set; 345 test crops show a plate whose text is also in train.	Tables 7 and 9 are partly measured on seen images, so they are optimistic for new photos (paper #23).
2	**The datasets are smaller than they look.** In our cleaning, only 185 of the 7,217 character crops survived: 4,894 were duplicates, 2,019 had masked digits, mosaics or incomplete labels, 119 were synthetic templates. Of 2,140 plate photos, 1,001 were duplicates (Figure 4).	"9,357 images" overstates how much distinct data the models saw.
3	**Max and mean are mixed.** The abstract and Table 17 give 284.31 ms (GPU) and 25.439 s (Jetson), which are the *maximum* times, next to the *mean* CPU time 2,311.53 ms (p.1, p.39).	The GPU mean is 117.16 ms; readers can mis-compare.
4	**"Edge deployment on Jetson"** is claimed as a contribution (p.6), but the Jetson ran on its CPU only, at 10.4 s per photo (p.17, p.36).	Not real time; the Jetson GPU was never used.
5	**Daylight only.** Own photos were all taken in daylight and sunny weather (p.6).	Our run: 38 % exact at night with their weights.
6	**Left-to-right sort only.** Bangladeshi plates have two lines; sorting by x alone can mix the rows.	Our decoder splits rows first.
7	**Small, self-chosen external set** (150 photos, "online and offline sources", p.32); 6 of the 150 are near-duplicates of other photos in our check.	One external set cannot show generalisation.
8	**OCR baseline not described:** the OCR engine and its settings are not named (p.37).	The 15.33 % cannot be reproduced.
9	**Text errors:** section 2.2 describes the original 2016 YOLOv1 (S×S grid, 448 px, fully connected layers), not YOLOv12 (p.10-13); "accuracy" uses true negatives, which do not exist in object detection (p.16); 2,251 vs 1,561 boxes (p.6 vs Fig. 3); duplicate class names (Table 3).	Shows the review was light; do not copy these parts.
10	**Unequal YOLO comparison:** Table 14 does not give the model size or settings of each version, and each was run once.	The v12 win (0.949 vs 0.936) may be noise or size.

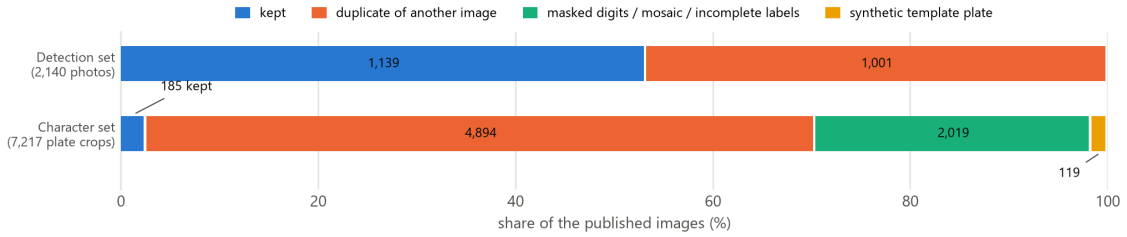


Figure 4. What our de-duplication and label check did to BLPD's published images (dataset/docs/dedupe_report.md). "Duplicate" = a Roboflow copy, a re-post, or the same photo in another public dataset; one copy of each cluster was kept. BLPD's plate boxes are loose, so its kept photos are used only as training hints.

7. What would break on our plates (and what we do about it)

Risk	Why	Our answer
Two-line plates read in the wrong order	x-sort mixes the rows when a bottom-row token starts left of the district word	row-aware decoder (cluster by y, then sort by x); the kit also logs the paper's pure x-sort (`xsort_exact_%`)
Rare districts missed	2 to 50 training boxes for some districts	merged data from many sources; synthetic plates for rare classes (proposal G6); district dictionary correction
Night, rain, blur, fog	training photos were daylight only	night photos in train, per-condition test sets, CLAHE and augmentation study (RQ4)
Vehicle not found	COCO has no class for CNG, rickshaw or leguna; close-up photos show no whole vehicle	skip the vehicle stage: the plate detector works on the whole frame (section 10, finding 3)
Too slow	three models, 8.5 FPS on an RTX 3050	two light models + TensorRT/ONNX/OpenVINO, recognition only for new tracks (RQ3, ≥ 10 FPS goal)
Wrong but plausible text	no check that the result is a legal plate	BRTA grammar check: District [Metro]-Letter NN-NNNN; invalid strings are flagged (O3)

8. What we ran and why

The notebook `paper02_kaggle.ipynb` rebuilds the paper's cascade on our **498R BanglaLPR All-in-One v1.0** dataset (merged from every public Bangla source, de-duplicated, grouped 70/15/15 split, seed 42, val/test with human labels only), and also runs the **authors' own weights**, so the two can be compared on the same photos.

Job	GPU	What it does	Time
`lpd_v12s`	0	YOLO12s plate detector, 60 epochs, 640, batch 16, SGD; train 8,574 photos (4,067 human + 4,507 auto boxes), val 896, test 1,153	186.7 min
`lpnr_v12n`	1	YOLO12n character detector, 70 epochs, 640, batch 16, 101 Bangla tokens; train 5,465 crops, val 536, test 467 crops (boxes) + 651 crops (plate text)	94.3 min
`cascade_ours`	0	YOLO12l (COCO) → our LPD → our LPNR on the 204 end-to-end photos, with and without the vehicle stage	0.8 min
`cascade_blpd_weights`	1	the same cascade with the authors' LPD_Model.pt and LPNR_Model.pt; also their LPNR on our 651 text crops	1.0 min
`speed_lpd_v12s`	0	batch-1 speed of the plate detector (PyTorch FP32/FP16, TensorRT FP16, CPU ONNX)	5.7 min

End-to-end test photos (204): 144 of the paper's own 150 external photos (6 were duplicates of other photos and were removed in our build) and 60 NightLPBD night photos. Every photo has its plate text typed in.

Differences from the paper: LPD 60 epochs instead of 100 (time budget); SGD instead of "auto" (Ultralytics 8.4's "auto" picks a different optimiser); model sizes s and n (the paper does not state its sizes); our class list has 101 tokens with the duplicates merged; our decoder splits two rows; our test data is different and harder (many sources, night, blur). Software: Ultralytics 8.4.170, PyTorch 2.11 + CUDA 12.8.

9. Results next to the paper

Metric	Paper (page)	Ours	Authors' weights in our run
LPD test precision / recall	98.6 / 96.5 (p.23)	96.7 / 96.2	-
LPD test mAP50	96.3 (p.23)	**98.9**	-
LPD test mAP50-95	68.5 (p.23)	46.2 (see 12.2)	-
LPNR test mAP50 / mAP50-95	94.9 / 76.7 (p.25)	92.6 / 63.9	-
LPNR on our 651 text crops: whole plate exact	-	**83.56**	72.50
Cascade: plate found in photo	100 (p.34)	93.14	88.24
Cascade: whole plate exact (all 204 photos)	95.33 (p.34, 150 photos)	**83.33**	78.43
Cascade: characters correct	98.82 (p.34)	**92.59**	88.18
Cascade: Levenshtein similarity	99.57 (p.34)	90.57	84.65
Cascade without vehicle stage: exact	-	82.84	77.94
Time per photo, mean	117.16 ms, RTX 3050 (p.35)	85.8 ms (T4)	91.2 ms (T4)
Time per photo, no vehicle stage	-	**31.5 ms (T4)**	36.3 ms (T4)
Training time LPD / LPNR	145 / 261 min, Colab T4 (p.17-24)	183 / 93 min, Kaggle T4	-

> **Read this table carefully.** The paper's rows were measured on the paper's data, ours on the 498R test split, which is larger, de-duplicated and harder. Only the last two columns are a true head-to-head: same photos, same code, different weights. The direct check of the paper's own numbers is the next table.

10. Four findings

Finding 1: the paper reproduces on its own photos

The authors' weights read 137 of their 144 external photos exactly (95.14 %) in our run; the paper reports 143 of 150 (95.33 %). Character accuracy is 98.93 % vs 98.82 %. This also shows that our cascade code, decoder and scoring are correct, because they give the paper's number when given the paper's models.

The check on the paper's own data

On BLPD's own data	Paper (page)	Our check	Where
Authors' weights, their external photos: whole plate exact	95.33 (p.34)	**95.14** (137 of 144)	Kaggle run, 19 Oct
Authors' weights, their external photos: characters correct	98.82 (p.34)	98.93	Kaggle run
Retrained with paper settings on BLPD's split: LPD test mAP50 / mAP50-95	96.3 / 68.5 (p.23)	97.2 / 68.1	local preview, 13 Oct (RTX 5060 Ti)
Retrained: LPNR test mAP50 / mAP50-95	94.9 / 76.7 (p.25)	96.7 / 80.0	local preview
Retrained cascade, 150 external photos: exact / characters	95.33 / 98.82 (p.34)	90.67 / 98.05	local preview

So the paper's printed numbers **are real on its own data**: its weights give the same external accuracy, and retraining with its settings gives the same detector scores. The official Kaggle version of this check (`paper02_original_check.ipynb`, about 7.5 h) has not been run yet.

Finding 2: it does not hold at night; diverse training data does

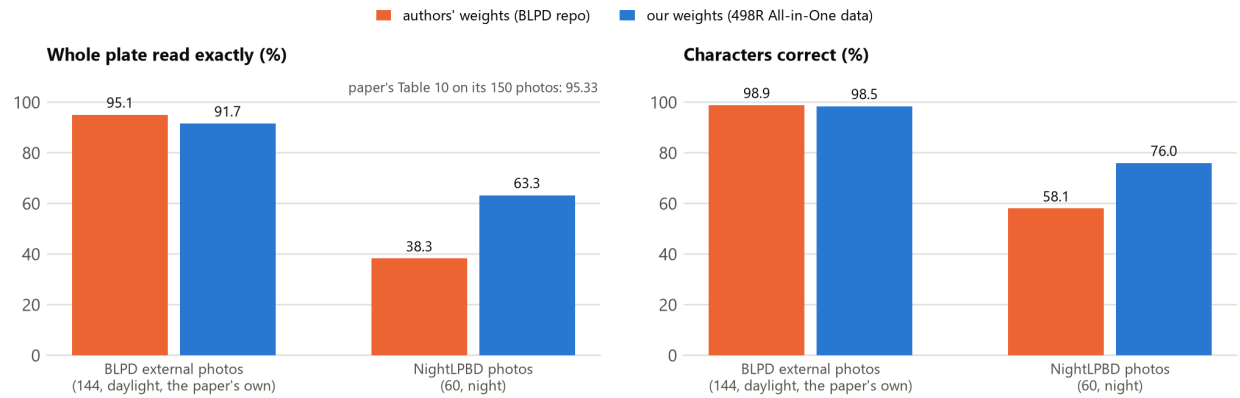


Figure 5. Same photos, same code, two sets of weights. Daylight (BLPD external): both models are good; the authors' weights are 3.5 points ahead on their own photos. Night (NightLPBD): the authors' weights collapse to 38.3 % exact; ours, trained on the merged data that includes night photos, reach 63.3 %.

At night the authors' model gets 58 % of characters right; ours 76 %. A 25-point gain from training data alone is our strongest evidence for RQ4 (robustness) and for building the merged dataset (O1). The authors' 3.5-point lead on their own photos is expected: their training data comes from the same sources and streets, while every near-duplicate of these test photos was removed from our training data.

Finding 3: the vehicle stage costs time and adds nothing

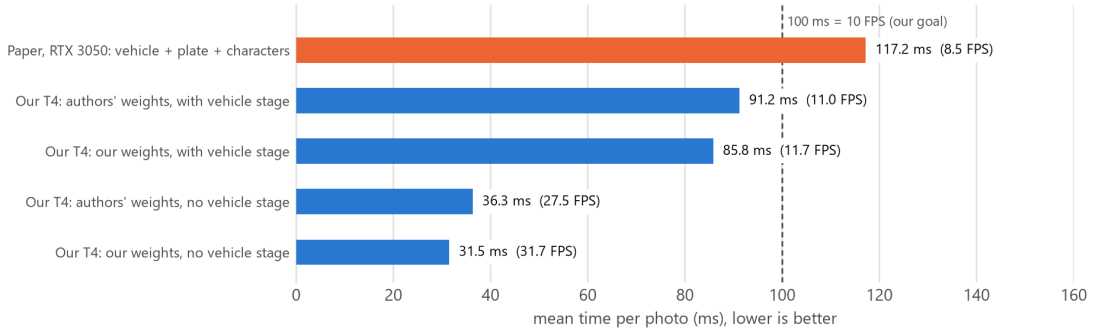


Figure 6. Mean time per photo. The paper's number is on an RTX 3050; ours on one Kaggle T4 (PyTorch, no TensorRT). Removing the COCO vehicle model cuts the time by 2.7×.

Pipeline (our weights, 204 photos)	Whole plate exact	Characters	ms per photo
vehicle → plate → characters (paper)	83.33 %	92.59 %	85.8
plate → characters (no vehicle stage)	82.84 %	93.57 %	31.5

The difference is one photo out of 204 (0.49 points); character accuracy is even higher without the vehicle stage. The same holds for the authors' weights (78.43 % vs 77.94 %, 91.2 vs 36.3 ms). The vehicle model is the largest of the three (YOLO12l) and, in the paper's own Table 12, it takes a third of the time. Dropping it is the cheapest way to reach our ≥ 10 FPS goal; on a T4 we are already at about 32 photos per second. The laptop CPU number is still to be measured.

Finding 4: where the models are weak

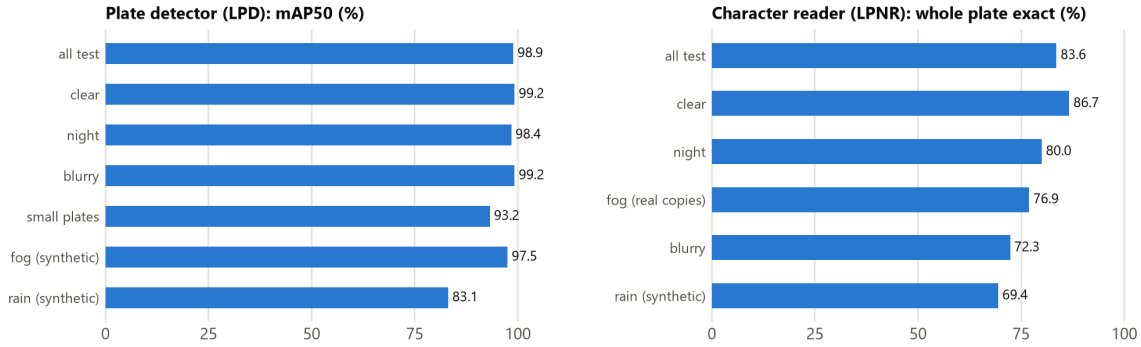


Figure 7. Per-condition test results of our two models (fixed team condition subsets; rain and detection-fog images are synthetic copies).

- **The plate detector** is almost perfect on clear, night and blurry photos (98-99 mAP50). It drops on small plates (93.2) and synthetic rain (83.1), and fires on 10 % of photos that contain no plate.
- **The character reader** is the weak link: 86.7 % of clear plates are read exactly, but only 72.3 % of blurry ones and 69.4 % with synthetic rain. This is where the proposal's grammar correction, enhancement and multi-frame voting should help.

11. Ours vs the proposal's targets

Proposal target	Our result now	Status
Detection mAP50 ≥ 95 %	98.9 % on the 1,153 test photos	met
Character accuracy ≥ 90 %	92.6 % on all 204 end-to-end photos (98.5 % daylight, 76.1 % night)	met overall, not at night
≥ 10 FPS full pipeline on standard hardware	about 32 photos/s on a T4 without the vehicle stage; laptop CPU not measured yet	on track; needs the laptop test
Results under standard vs duplicate-free split (RQ1)	BLPD audit: 44.1 % / 37.4 % of test images duplicated in train	evidence ready; a leaky-vs-fair BLPD run is a next step

12. Health check: is the training OK?

> **Yes, the run is healthy.** All five jobs ended with `code=0`. LPD took 186.7 min (cap 5 h) and LPNR 94.3 min (cap 4 h), so both ran all their epochs. The dataset downloaded and unpacked in under a minute with the expected counts. Both cascades processed all 204 photos. Total run: 11,723 s (3.3 h) on Tesla T4 x2.

12.1 Do not quote the "7.30 FPS" row

The table row "LPD FPS, T4 FP16, batch 1 = 7.30" is the *PyTorch FP16* number, not TensorRT, and it cannot be right: in the cascade, the same detector *plus* the character reader took only 31.5 ms per photo (≈ 32 FPS) on the same GPU. It is a measurement artefact (the speed loop may time something besides the model). The real speed breakdown (FP32, FP16, TensorRT FP16, CPU ONNX) is in `runs/speed_lpd_v12s/results.json` inside the outputs zip. Until then, quote the cascade times (85.8 / 31.5 ms).

12.2 LPD mAP50-95 is low (46.2 vs the paper's 68.5)

mAP50 is 98.9, so the detector *finds* almost every plate; mAP50-95 measures how *tight* the box edges are. Our test labels come from seven sources (KUET 540, ridoykj 274, LPDB-A 205, NightLPBD 60, others 74) that draw plate boxes in different styles (tight to the characters, or around the whole metal plate), and the 4,507 auto-labels in train add a third, averaged style. The model cannot match every style at IoU 0.9. The auto-label detector of our dataset build scored only 57 mAP50-95 on validation for the same reason. **This is our best explanation, not yet proven.** Check: train once on human labels only (`data_human.yaml`) and score mAP50-95 per source. For plate reading this does not matter much: the plate crop is padded by 8 % before reading.

12.3 Two numbers that are not a fair test of the authors' method

- "Authors' LPNR on our text crops: 72.50 %" compares their model on *our* crops and label style against our model trained on that style (83.56 %). It shows domain shift, not that their method is worse.
- The first results table mixes datasets (section 9 warning). The fair comparisons are the same-photo columns and Figure 5.

12.4 The outputs were not fully downloaded

The Kaggle log and notebook are saved, so every number in this report is safe. But `paper02_outputs.zip` (139.1 MB, with weights) stopped downloading (a `.crdownload` file appeared and then vanished), and `paper02_results.zip` (2.3 MB) was not downloaded. Without them we cannot see: which 14 photos had no plate found, the x-sort vs row-decoder accuracy (`xsort_exact_%`), the speed breakdown, curves and confusion matrices. Download both from the notebook's **Output** tab before the Kaggle version expires.

12.5 Smaller things

- LPD trained 60 epochs, not 100. LPNR mAP50 (92.6) is 2.3 points under the paper, but on a de-duplicated test set; the paper's test set had 37.4 % duplicates.
- Character accuracy in the cascade counts photos with no plate found as fully wrong, so it is a strict number.
- One seed only; repeat with 2 more seeds before the final report.

13. How to use this paper in our research

Our research part	How paper #2 is used	Evidence we now have
Stage 2: character-level recognizer (O3)	design source: one YOLO detects digits, letters, "Metro" and whole district words; no OCR engine	their idea gives 92.6 % characters on our 204 test photos
Baseline for every table	the authors' public weights, run on our photos with our code	same-photo comparison: 95.1 vs 91.7 (day), 38.3 vs 63.3 (night)
RQ1: duplicate-free split	BLPD is a second leaky dataset next to KUET (paper #23)	44.1 % of LPD test and 37.4 % of LPNR test duplicated in train
RQ2: ≥ 90 % characters with grammar correction	their x-sort decoder is the "without correction" baseline	92.6 % already; row decoder + grammar should add on top
RQ3 / O4: real time	their 3-model cascade is the slow reference (8.5 FPS RTX 3050, 2.3 s laptop CPU)	no vehicle stage: 2.7× faster, same accuracy
RQ4: robustness	their daylight-only model shows what happens without diverse data	night: +25 points with our merged data
O2: detector choice	their YOLO v8-v12 comparison (Table 14) is a starting point, but on leaky data	redo it on our split with equal model sizes
Thesis writing	Introduction (state of the art), Related work (their Table 1 lists 14 Bangla systems with class counts), Method, Experiments	BibTeX at the end

14. What benefit do you get from it?

- **A strong, citable baseline.** A 2026 peer-reviewed journal paper with public weights. Beating it, or explaining where it fails, is a clear contribution that examiners understand.
- **Proof that our evaluation code is right.** Our cascade gives the paper's own number (95.1 vs 95.3) with the paper's weights. Any later number from the same code is therefore trustworthy.
- **A ready story for the faculty:** best published system → reproduced → its blind spots (night, leakage, speed) → our fixes. Each step has a number.
- **Design decisions with evidence** instead of opinion: OCR-free recognition (95.33 vs 15.33 in the paper), no vehicle stage (finding 3), a clean class list (duplicate classes found), a two-row decoder.
- **Support for our dataset work.** Figure 4 and the leakage audit show why the merged, de-duplicated benchmark (O1) is needed and why our numbers are lower than published ones: they are honest.
- **The authors' future work is our project.** They list low light, weather, blur, occlusion, imbalance and lightweight models (p.41); our proposal covers all of them.
- **Reusable assets:** their district/letter class scheme (mapped to our 101 tokens), their metrics (exact, character, Levenshtein), and their latency reporting style (per stage, mean and max, per device).

A paragraph you can adapt for the related-work chapter

```
Sarker and Alam [ref] cascade three YOLOv12 models (a COCO vehicle detector, a plate
detector and a 104-class character detector that treats district names as single
objects) and report 95.33% full-plate accuracy on 150 external images, against 15.33%
for an OCR pipeline. Their data are daylight-only and split by image; in our audit
44.1% of their detection test images and 37.4% of their character test images have
a near-duplicate in training. Re-running their released weights, we obtain 95.14% on
their external images but 38.3% on night images, and we find that the vehicle stage
can be removed with no loss in accuracy at 2.7x lower latency.
```

15. What to show the faculty (7 slides)

#	Slide title	What is on it	Source in this report
1	The problem	plate parts (city, Metro, letter, numbers); why OCR fails on Bangla; 14-60 classes in older work	Figure 1, section 1
2	The paper's method	three YOLOv12 models in a cascade, 104 classes, left-to-right decoding	Figure 2 (top row)
3	The paper's results	95.33 % exact vs 15.33 % OCR; LPD mAP50 0.963, LPNR 0.949; 117 ms on RTX 3050, 2.3 s on CPU	section 5 table
4	We reproduced it	authors' weights on their photos 95.14 vs 95.33; retrained detectors match Tables 7 and 9	section 9, second table
5	But: its data leaks	44 % of test photos duplicated in train; 185 of 7,217 character crops survive cleaning	Figure 4, weakness 1-2
6	Our run on clean data	night 38 → 63 % (Figure 5); no vehicle stage = 2.7× faster (Figure 6); 92.6 % characters	Figures 5 and 6
7	How it shapes our system	our pipeline (Figure 2 bottom row) and the next steps	Figure 2, section 18

> **Keep it honest on slide 6:** say "on the same photos" when you compare with the authors' weights, and "on our test split" for the rest. Do not put 7.30 FPS or the 46.2 mAP50-95 on a slide without the explanation from section 12.

16. What to explain: a 3-minute script

"This paper is the strongest recent work on Bangladeshi plates: a 2026 PLOS One article from Khulna University. They chain three YOLOv12 models: one finds the vehicle, one finds the plate, and one finds every character, and even whole district names like 'Dhaka', as separate objects. Because of that they need no OCR engine, which is good, because OCR fails on Bangla conjunct letters. They report 95 percent of plates read exactly on 150 external photos, against 15 percent for OCR.

We downloaded their released weights and ran them on their own photos with our code. We got 95.1 percent, so their result is real and our code is correct. We also retrained their models with their settings and got the same detector scores.

Then we looked at their data. 44 percent of their plate test photos and 37 percent of their character test images also appear, in near-identical form, in their training set, so their test scores are optimistic. After removing duplicates and bad labels, only 185 of their 7,217 character images were unique and usable.

On night photos their model reads only 38 percent of plates. Our model, trained on our merged and de-duplicated dataset, reads 63 percent: 25 points better with the same method. We also found that their first model, the vehicle detector, can be removed: accuracy stays the same and the pipeline becomes 2.7 times faster, about 32 photos per second on a T4.

So we keep their best idea, the character-level YOLO with word classes, and change the rest: no vehicle stage, a two-row decoder, a BRTA grammar check, tracking for video, and a fair, de-duplicated benchmark. That is exactly the plan in our proposal."

17. Questions the faculty may ask

Q1. Why not just use their model?
It was trained on daylight photos only and gets 38 % at night; it uses a slow vehicle stage; its class list has duplicate names; and its test scores are inflated by duplicates. We keep its idea, not its weights.

Q2. Your numbers are lower than theirs. Is your model worse?
Not necessarily. Our test set is different: larger, many sources, night and blur, and no duplicates of training images. On the same photos the two models are close in daylight (95.1 vs 91.7) and ours is far better at night (63.3 vs 38.3).

Q3. Why is your detector's mAP50-95 only 46?
It finds the plates (mAP50 98.9) but its box edges differ from labels drawn in several styles by different datasets. We will confirm this with a human-label-only run and per-source scores (section 12.2).

Q4. Is removing the vehicle stage safe?
On 204 photos it changed the result for one photo, and character accuracy went up. It also avoids missing vehicles that COCO does not know (CNG, rickshaw) and close-ups where no whole vehicle is visible.

Q5. How do you know your evaluation code is correct?
With the authors' weights it reproduces their Table 10 (95.14 vs 95.33 % exact, 98.93 vs 98.82 % characters).

Q6. Is their 15 % OCR baseline fair?
We cannot tell: the paper does not name the OCR engine or its settings. Paper #6's kit runs EasyOCR on our data, so we will report our own OCR baseline.

Q7. What is Levenshtein similarity and why is it so high?
It gives partial credit: one wrong character in a 12-character plate still scores about 92 %. That is why 99.57 % similarity sits next to 95.33 % exact. We report exact match as the main number.

Q8. Why YOLOv12? Is it better than YOLOv8?
The paper found v12 best (0.949 vs 0.936), but without model sizes and with one run on leaky data. Objective O2 repeats the comparison on our split with equal sizes and several seeds.

Q9. How will you handle two-line plates?
The paper sorts all boxes left to right. Our decoder first splits the boxes into two rows by their vertical position, then sorts each row; the kit stores both results, so we can show the gain.

Q10. Is it real time?
The paper's cascade: 8.5 FPS on an RTX 3050 (mean), 0.4 FPS on a laptop CPU. Ours without the vehicle stage: about 32 photos per second on a T4. The laptop test with ONNX/OpenVINO is next (target ≥ 10 FPS).

18. Next steps (in order)

1. **Download the outputs today**: `paper02_outputs.zip` (139.1 MB) and `paper02_results.zip` (2.3 MB) from the Kaggle Output tab. Then run `python add_results.py paper02_outputs.zip --member "Rokib Hasan Oli"` in `docs/results`; it fills the notes and the team table.
2. **Open** `e2e_preds_*.csv` in the zip: list the photos with no plate found and compare the x-sort column with the row decoder (Q9).
3. **Run** `paper02_original_check.ipynb` on Kaggle (about 7.5 h) to make the reproduction of Tables 6-10 official.
4. **LPD box check:** one run on human labels only and mAP50-95 per source (section 12.2); one run with 100 epochs like the paper.
5. **Speed on a laptop:** plate detector + character reader exported to ONNX/OpenVINO, measured on a normal laptop CPU (≥ 10 FPS target).
6. **Night:** more night training data, augmentation and CLAHE; score on NightLPBD and the night condition subset.
7. **Grammar correction (O3):** district dictionary + BRTA pattern; report exact match with and without it.
8. **Detector and reader choice (O2):** YOLOv8n / YOLO11n / YOLO12n with equal settings and 3 seeds on our split.
9. **Leaky vs fair BLPD:** repeat the paper #23 experiment on BLPD (44.1 % leaky test).

19. GitHub and Drive

After the download, use the same layout as paper #23 and Md. Yousuf's `weights/paper03/Md.Yousuf/`:

Where	What
`CSE498R / weights / paper02 / MD.Rokib.Hasan.Oli /`	README.md (main results + limits from this report), best_lpd_v12s.pt, best_lpnr_v12n.pt, paper02_results.zip, results_vs_paper.md, this PDF (optional)
Team Drive `498R / weights / paper02 /`	paper02_outputs.zip (139.1 MB, too big for the GitHub web upload limit of 25 MB per file)

Web upload: open the repo's `weights` folder → Add file → Upload files → drag the `paper02` folder → commit message "Add paper #2 results: YOLO12 cascade on 498R data (Rokib)".

Appendix: where every number comes from

Number	Source
paper numbers	PLOS One e0355858, pages given in the text
our training, cascade, condition results	After `train/paper02-r.log` and `paper02-r.ipynb` (Kaggle run, outputs of cells 15-19)
per-source cascade numbers (blpd_ext, nightlpbd)	the `by_source` lines printed in cell 17 of the notebook
retraining on BLPD's own split	local preview of 13 Oct 2026 in the paper #2 notes (RTX 5060 Ti, Ultralytics 8.4.170)
leakage (44.1 %, 37.4 %, 345 crops)	`dataset/docs/leakage_audit.md`
cleaning (185 of 7,217; 1,001 of 2,140; 6 external duplicates)	`dataset/docs/dedupe_report.md`
proposal targets and RQs	`Proposal/src/proposal_content.py`

```
@article{sarker2026cascaded,
  title   = {Cascaded {YOLO}-based approach for {Bangladeshi} license plate
             identification and number recognition},
  author  = {Sarker, Proshenjit and Alam, S. M. Shamsul},
  journal = {PLOS One},
  volume  = {21}, number = {8}, pages = {e0355858}, year = {2026},
  doi     = {10.1371/journal.pone.0355858}
}
```